

Guia paso a paso — IDaaS, API Manager y Colas (EFT S9, Grupo 13)

Configuracion de los tres servicios cloud del EFT. Reemplaza los **TU_...** por los valores de tu laboratorio. Los valores reales del Grupo 13 estan al final.

PARTE A — Identity as a Service (Azure AD B2C)

A1. Tenant y aplicacion

1. <https://portal.azure.com> → cambia al tenant B2C (**duocgrupo13**).
2. **Azure AD B2C** → **App registrations** → **New registration**: nombre **eft-cursos-grupo13** (o reutiliza la app existente). Anota **Application (client) ID** y **Directory (tenant) ID**.
3. **Authentication**: marca *Access tokens* e *ID tokens*. Agrega **Redirect URIs**:
 - **SPA**: <http://localhost:8080> (frontend local) y la URL del frontend en la nube.
 - **Web**: <https://jwt.ms> y <https://oauth.pstmn.io/v1/callback> (para pruebas en Postman).

A2. Rol como custom claim (ESTUDIANTE / INSTRUCTOR)

1. **User attributes** → **Add**: **Role** (String). Azure lo expone como **extension_Role**.
2. **User flows** → **New user flow** → **Sign up and sign in** → **B2C_1_signupsignin**. En **Application claims** marca **Role**, Display Name y Email.
3. **Users**: crea 2 usuarios de prueba y asigna el atributo **Role**: uno **ESTUDIANTE** y otro **INSTRUCTOR**.

A3. Scope (para el access token)

1. **Expose an API** → **Add a scope** (acepta el App ID URI). Crea el scope de acceso.
2. **Certificates & secrets** → **New client secret** (para Postman). Copia el Value.

A4. Datos que consumen el backend y el frontend

```
Issuer:  https://<tenant>.b2clogin.com/<tenant-id>/v2.0/
JWKS:
https://<tenant>.b2clogin.com/<tenant>.onmicrosoft.com/B2C_1_signupsignin/discovery/v2.0/keys
Claim:   extension_Role   (ESTUDIANTE / INSTRUCTOR)
```

En el backend: variables **AZURE_B2C_ISSUER**, **AZURE_B2C_JWK_SET_URI**, **AZURE_B2C_ROLES_CLAIM**.

En el frontend: **frontend/config.js** (clientId, authority, knownAuthority, scopes).

PARTE B — API Manager (AWS API Gateway)

Ten los 2 microservicios desplegados en la EC2 (http://IP_EC2:8081 y [:8082](http://IP_EC2:8082)).

B1. Crear la HTTP API

1. **API Gateway** → **Create API** → **HTTP API** → **Build**. Nombre **api-eft-cursos-grupo13**.

B2. Autorizador JWT

1. **Authorization** → **Manage authorizers** → **Create** → tipo **JWT**, nombre **b2c-jwt**.
2. **Issuer URL** = el issuer de B2C; **Audience** = Application (client) ID.

B3. Registrar TODOS los endpoints (Routes → Create) y asociar el autorizador

courses-service (integración HTTP http://IP_EC2:8081):

Metodo / Ruta
POST /api/cursos
GET /api/cursos
GET /api/cursos/{id}
PUT /api/cursos/{id}
DELETE /api/cursos/{id}
POST /api/cursos/{id}/material
GET /api/cursos/{id}/material
POST /api/inscripciones
GET /api/inscripciones/mias
POST /api/calificaciones
GET /api/calificaciones/mias

bff-service (integración HTTP http://IP_EC2:8082):

Metodo / Ruta
POST /api/bff/cola/producir
POST /api/bff/cola/consumir
GET /api/bff/cola/errores

En HTTP API la **URI de integración es solo el host** (<http://IP:8081>); el gateway agrega el path. Asocia el autorizador **b2c-jwt** a cada ruta.

B4. Desplegar

1. **Deploy** → **Stages** → **Create** (**prod** o **\$default**). Copia la **Invoke URL**.
2. Actualiza **frontend/config.js** (**apiCursos/apiBff**) con la Invoke URL.

PARTE C — Colas (RabbitMQ)

C1. Despliegue

RabbitMQ corre en un contenedor Docker (servicio **rabbitmq** del **docker-compose.yml**, puertos 5672 y 15672). Consola: <http://localhost:15672> (guest/guest).

C2. Topología (declarada por el bff-service al arrancar)

Elemento	Nombre
----------	--------

Elemento	Nombre
Exchange	<code>eft.exchange</code> (direct)
Cola principal	<code>eft.col.a.principal</code> (con DLX → <code>eft.dlx</code> / rk <code>eft.rk.error</code>)
Dead Letter Exchange	<code>eft.dlx</code> (direct)
Cola de errores (DLQ)	<code>eft.col.a.errores</code>

C3. Productor y consumidor (Java)

- **Productor** (`ColaProducer`): `POST /api/bff/cola/producir` publica un `MensajeEft` en la cola principal.
- **Consumidor** (`ColaConsumerService`): `POST /api/bff/cola/consumir` procesa los mensajes; si uno falla hace `basicNack(requeue=false)` y RabbitMQ lo **dead-letters** a `eft.col.a.errores` via el DLX. `GET /api/bff/cola/errores` muestra la DLQ.

Flujo:

```

producir → eft.exchange → eft.col.a.principal → (consumir)
    OK → procesado
    FALLO → nack(no requeue) → eft.dlx → eft.col.a.errores (DLQ)

```

Valores reales del Grupo 13

Parametro	Valor
Tenant	duocgrupo13.onmicrosoft.com
Tenant ID	3f8624f7-de01-47f3-9fd0-f834f87dc061
Application (client) ID / audience	259dff0d-8d49-41ef-8f85-18bebb472ec0
User flow	B2C_1_signupsigin
Issuer	https://duocgrupo13.b2clogin.com/3f8624f7-de01-47f3-9fd0-f834f87dc061/v2.0/
JWKS	https://duocgrupo13.b2clogin.com/duocgrupo13.onmicrosoft.com/B2C_1_signupsigin/discovery/v2.0/keys
Claim de rol	extension_Role (ESTUDIANTE / INSTRUCTOR)
Bucket S3	eft-cursos-grupo13